

吴锡华答辩闭环：测试、运行支撑与使用说明

1. 先记住自己的定位

你这部分的核心不是“讲编译原理算法”，而是守住三件事：

1. 你们怎么验证编译器不是只靠平台样例碰运气。
2. 成功样例和错误样例各怎么测试。
3. 使用说明和构建流程怎样保证别人拿到项目能复现。

最短自我介绍可以直接说：

我主要参与的是测试、运行支撑与使用说明整理。核心工作是把编译器的构建、运行、成功样例验证、错误样例验证和严格 C11 检查串成一套可复现流程，确保项目不仅能过平台，还能在本地稳定复验。

2. 现场先打开哪些文件

建议只记这 3 个文件，够用了：

1. [course_tests/run_course_tests.ps1](#)
2. [course_tests/run_strict_c_checks.ps1](#)
3. [submission/06_user_guide/user_guide.typ](#)

打开顺序建议：

1. 先看 `run_course_tests.ps1`，讲“成功样例和错误样例怎么验证”。
 2. 再看 `run_strict_c_checks.ps1`，讲“为什么还要做严格 C11 检查”。
 3. 最后看 `user_guide.typ`，讲“如何把构建和运行方法交付给老师和同学”。
-

3. 老师最可能问什么

问题 A：你们除了平台 95/95，还做了什么测试？

先打开：

- [course_tests/run_course_tests.ps1:21](#)
- [course_tests/run_strict_c_checks.ps1:17](#)

回答模板：

除了平台 95/95，我们还做了三层本地验证：

第一层是课程自建测试集 `course_tests`，分成功样例和错误样例；

第二层是 `user_tests` 本地回归样例；

第三层是严格 C11 检查，验证生成出来的 `.c` 文件不仅能跑，还能通过更严格的标准编译。

问题 B：成功样例怎么验证？

先打开：

- [course tests/run_course_tests.ps1:27](#)
- [course tests/run_course_tests.ps1:42](#)

回答模板：

成功样例的验证链路是：

Pascal-S 源程序先交给编译器生成 C 文件；

再用 `gcc` 把生成的 C 编译成可执行文件；

最后运行并把输出和 `.out` 期望结果比对。

这样不只验证了编译器本身，也验证了生成代码是可编译、可运行的。

问题 C：错误样例怎么验证？

先打开：

- [course tests/run_course_tests.ps1:74](#)

回答模板：

错误样例不要求继续生成可执行文件，而是要求编译器在失败时输出正确的诊断信息。

每个错误样例都配一个 `.err.txt`，里面保存关键子串，比如错误类型、字段名、箭头定位等。测试脚本会检查这些关键片段是否都出现在实际错误输出里。

如果老师追问“为什么不做整段字符串完全相等”：

因为行号或格式细节可能会有小变化，用关键字串匹配更稳，同时又能覆盖真正重要的诊断信息。

问题 D：为什么还要做严格 C11 检查？

先打开：

- [course tests/run_strict_c_checks.ps1:22](#)
- [course tests/run_strict_c_checks.ps1:32](#)

回答模板：

因为我们做的是 Pascal-S 到 C 的源到源编译器，所以不能只看“编译器跑没跑”，还要看生成出来的 C 是否规范。

严格 C11 检查会用 `-std=c11 -pedantic-errors -Wall -Wextra -Werror` 去编译生成代码，这可以帮助发现非标准写法或潜在问题。

这句很值钱：

这说明我们不是只追求平台通过率，还在验证目标代码质量。

问题 E：现场怎么运行演示？

建议他不要自己主讲演示，但要知道流程。

可以答：

现场演示主要是两条：一条成功样例，一条错误样例。
成功样例会展示 Pascal-S 先编译成 C，再编译运行得到结果；
错误样例则展示 `Semantic Error` 和 `caret diagnostics` 定位输出。

问题 F：程序使用说明写了什么？

先打开：

- [submission/06_user_guide/user_guide.typ:32](#)
- [submission/06_user_guide/user_guide.typ:73](#)
- [submission/06_user_guide/user_guide.typ:97](#)
- [submission/06_user_guide/user_guide.typ:174](#)

回答模板：

使用说明主要整理了四类内容：
第一是程序概述和支持功能；
第二是 Windows/Linux/平台环境下的构建方法；
第三是编译器的命令行使用方式和生成代码后如何继续运行；
第四是错误提示、测试方法和平台提交注意事项。

4. 最该会指给老师看的 5 段代码/文档

4.1 `run_course_tests.ps1` 成功样例主循环

位置：

- [course_tests/run_course_tests.ps1:27](#)

怎么讲：

这段验证成功样例：先生成 C，再用 `gcc` 编译，再运行并对比输出。

4.2 `run_course_tests.ps1` 错误样例主循环

位置：

- [course_tests/run_course_tests.ps1:74](#)

怎么讲：

这段验证错误样例：期待编译失败，并检查错误输出是否包含预期诊断片段。

4.3 Normalize-Text()

位置:

- [course_tests/run_course_tests.ps1:9](#)

怎么讲:

这里对换行和末尾空白做归一化，避免不同环境下输出格式轻微差异影响结果比较。

4.4 严格 C11 检查命令

位置:

- [course_tests/run_strict_c_checks.ps1:32](#)

怎么讲:

这里直接用严格的 C11 编译选项编译生成代码，确保它不仅“能过”，而且比较规范。

4.5 使用说明中的构建和运行章节

位置:

- [submission/06_user_guide/user_guide.typ:73](#)
- [submission/06_user_guide/user_guide.typ:97](#)

怎么讲:

这里把构建命令、运行命令和平台提交注意事项都整理成正式文档，保证别人拿到项目后能复现。

5. 如果老师一个点一个点追问，怎么答

5.1 你们测试覆盖了哪些功能点？

回答:

覆盖了基础表达式、数组、函数/过程调用、控制流、`record` 扩展、`repeat-until` 扩展，以及错误样例中的参数错误、字段错误、类型错误和语法错误。

5.2 为什么测试里既有 `course_tests` 又有 `user_tests`？

回答:

`course_tests` 是正式课程自建测试集，结构更完整，分成功和错误样例；`user_tests` 更像日常开发时的本地快速回归样例。

5.3 平台脚本和本地脚本有什么区别？

回答：

本地脚本主要服务构建、回归和严格 C11 检查；平台环境则要把最终可执行文件放到指定目录，并按平台约定调用。使用说明里把两种环境分开写了。

6. 老师如果让你“讲设计思路”，你就这么收

我们这一层的设计思路是把“能构建、能运行、能验证、能交付”做成一套完整闭环，而不是只在平台上点一下通过。成功样例验证程序结果，错误样例验证诊断质量，严格 C11 检查验证生成代码规范性，使用说明则保证整个流程可复现。

7. 最短保底版：30 秒回答

如果他非常紧张，只背这段也够过关：

我主要参与的是测试、运行支撑和使用说明整理。我们有课程自建测试集，成功样例会走“Pascal-S -> C -> gcc -> 运行结果对比”，错误样例会检查诊断输出；另外还做了严格 C11 检查，确保生成代码规范。相关代码主要在 `run_course_tests.ps1`、`run_strict_c_checks.ps1` 和 `user_guide.typ`。

8. 最容易说错的 4 个点

1. 不要把测试说成只有平台 95/95。
正确说法：还有本地课程测试、自建错误样例和严格 C11 检查。
2. 不要把错误样例说成“编译失败就行”。
正确说法：要检查诊断输出是否包含预期关键片段。
3. 不要把使用说明说成只是文档。
正确说法：它整理了构建、运行、测试和平台提交流程，是交付的一部分。
4. 不要忘记强调“生成代码质量”。
这是严格 C11 检查存在的意义。

9. 吴锡华只需要记住的文件定位

最推荐只记下面这些：

- [course_tests/run_course_tests.ps1:9](#)
- [course_tests/run_course_tests.ps1:27](#)
- [course_tests/run_course_tests.ps1:74](#)
- [course_tests/run_strict_c_checks.ps1:22](#)
- [course_tests/run_strict_c_checks.ps1:32](#)
- [submission/06_user_guide/user_guide.typ:73](#)
- [submission/06_user_guide/user_guide.typ:174](#)

这 7 处足够覆盖他的大部分答辩问题。